El Problema del Camino Más Corto

Algoritmo de Dijkstra

Introducción

En la vida real, muchos problemas pueden modelarse como la búsqueda del camino más corto entre dos puntos en una red. Desde los sistemas de navegación GPS que calculan la ruta más rápida entre dos ciudades, hasta los protocolos de enrutamiento en Internet que determinan cómo viajan los paquetes de datos, el problema del camino más corto es uno de los problemas fundamentales de la informática aplicada.

Formalmente, dado un grafo ponderado G = (V, E) donde V es el conjunto de vértices y E es el conjunto de aristas con pesos w(u, v) ≥ 0, el problema consiste en encontrar el camino de menor costo total desde un vértice origen s hasta un vértice destino t.

En 1956, el científico holandés Edsger W. Dijkstra diseñó un algoritmo eficiente para resolver este problema cuando todos los pesos de las aristas son no negativos. El algoritmo, conocido como algoritmo de Dijkstra, utiliza una estrategia voraz (greedy) que garantiza encontrar la solución óptima global a partir de decisiones locales óptimas.

Aplicaciones Prácticas

- GPS y sistemas de navegación (Google Maps, Waze)
- Enrutamiento de paquetes en redes de computadoras (protocolos OSPF, IS-IS)
- Redes sociales: cálculo del grado de separación entre usuarios
- Logística y cadenas de suministro: optimización de rutas de entrega
- Videojuegos: pathfinding de inteligencia artificial (IA)
- Diseño de circuitos electrónicos: ruteo de señales

Conceptos

Conceptos Fundamentales de Grafos

Término	Definición
Grafo G = (V, E)	Estructura matemática compuesta por un conjunto de vértices V y un conjunto de aristas E que los conectan.
Vértice (nodo)	Unidad fundamental del grafo, representa una entidad (ciudad, equipo, estado).
Arista (arco)	Conexión entre dos vértices. Puede ser dirigida (digrafo) o no dirigida.
Peso (costo)	Valor numérico asignado a una arista que representa su costo, distancia o tiempo.
Camino	Secuencia de vértices v• , v, , ..., vk donde cada par consecutivo está unido por una arista.
Camino más corto	Camino entre dos vértices cuya suma de pesos es mínima.
Grafo ponderado	Grafo donde cada arista tiene un peso asociado.
Distancia	Suma total de pesos en el camino más corto entre dos vértices.

Representación de Grafos

Existen dos formas principales de representar un grafo en memoria:

Matriz de Adyacencia

Es una matriz bidimensional M de tamaño n × n donde M[i][j] contiene el peso de la arista entre los vértices i y j. Si no existe arista, M[i][j] = ∞ (o se representa con 0 o Integer.MAX_VALUE en la práctica).

• Ventaja: acceso en O(1) para verificar adyacencia. • Desventaja: ocupa O(V²) memoria.

Lista de Adyacencia

Cada vértice almacena una lista de sus vecinos junto con el peso de la arista que los conecta. Es la representación preferida para grafos dispersos.

• Ventaja: ocupa O(V + E) memoria. • Desventaja: verificar adyacencia puede tomar O(grado) tiempo.

Algoritmo de Dijkstra

El algoritmo de Dijkstra es un algoritmo de búsqueda de caminos más cortos que opera bajo el principio greedy: en cada paso selecciona el vértice no visitado con la menor distancia acumulada conocida y actualiza las distancias de sus vecinos (proceso llamado relajación de aristas).

Condición necesaria: todos los pesos de las aristas deben ser no negativos. Para grafos con pesos negativos se debe usar el algoritmo de Bellman-Ford.

Invariante del algoritmo:

En cada iteración, los vértices ya visitados tienen calculada su distancia mínima definitiva desde el origen. Esta propiedad garantiza la corrección del algoritmo.

Fórmula de Relajación:

Si dist[u] + w(u, v) < dist[v], entonces dist[v] = dist[u] + w(u, v) y pred[v] = u

Donde dist[u] es la distancia acumulada hasta el vértice actual u, w(u, v) es el peso de la arista (u, v), y dist[v] es la distancia actual al vecino v.

Planteamiento del Problema

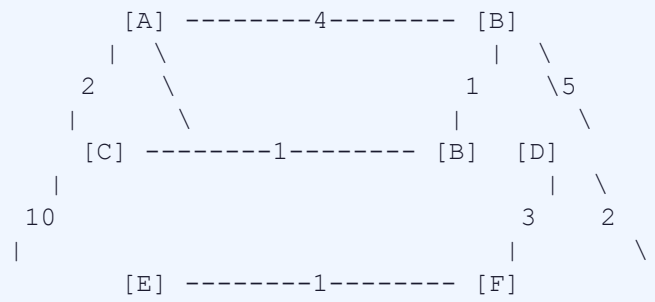
Se tiene un grafo no dirigido y ponderado con 6 vértices: A, B, C, D, E y F. Las aristas con sus respectivos pesos son las siguientes:

Arista	Vértice Origen	Vértice Destino	Peso	Descripción
(A, B)	A	B	4	Conexión directa A-B
(A, C)	A	C	2	Conexión directa A-C
(B, C)	B	C	1	Conexión directa B-C
(B, D)	B	D	5	Conexión directa B-D
(C, E)	C	E	10	Conexión directa C-E
(D, E)	D	E	3	Conexión directa D-E
(D, F)	D	F	2	Conexión directa D-F
(E, F)	E	F	1	Conexión directa E-F

Representación Visual del Grafo

El grafo puede representarse esquemáticamente de la siguiente manera (los números sobre las aristas indican los pesos):

Representación del Grafo G = (V, E)



V = {A, B, C, D, E, F} Objetivo: encontrar el camino más corto de A a F

Matriz de Adyacencia

La siguiente matriz muestra los pesos entre vértices. El símbolo ∞ representa la ausencia de arista directa.

	A	B	C	D	E	F
A	—	4	2	∞	∞	∞
B	4	—	1	5	∞	∞
C	2	1	—	∞	10	∞
D	∞	5	∞	—	3	2
E	∞	∞	10	3	—	1
F	∞	∞	∞	2	1	—

Desarrollo Paso a Paso

A continuación se traza manualmente el algoritmo de Dijkstra sobre el grafo definido. El vértice origen es A y el destino es F.

Paso 1: Representar el Grafo (Lista de Adyacencia)

Construimos la lista de adyacencia del grafo. Cada vértice almacena sus vecinos y el peso de la arista:

Vértice	Lista de Adyacencia (vecino, peso)
A	`(B, 4),  (C, 2)`
B	`(A, 4),  (C, 1),  (D, 5)`
C	`(A, 2),  (B, 1),  (E, 10)`
D	`(B, 5),  (E, 3),  (F, 2)`
E	`(C, 10),  (D, 3),  (F, 1)`
F	`(D, 2),  (E, 1)`

Paso 2: Inicialización del Algoritmo

Se inicializan los arreglos de distancias, visitados y predecesores. El vértice origen es A con distancia 0; todos los demás tienen distancia ∞.

Vértice	dist[]	visitado[]	pred[]	Observación
A	0	false	null	Origen del camino
B	∞	false	null	Sin distancia conocida
C	∞	false	null	Sin distancia conocida
D	∞	false	null	Sin distancia conocida
E	∞	false	null	Sin distancia conocida
F	∞	false	null	Sin distancia conocida

Paso 3: Tabla Maestra de Iteraciones del Algoritmo

La siguiente tabla muestra el estado completo de los arreglos dist[] y pred[] al finalizar cada iteración. Los valores en verde indican actualizaciones respecto a la iteración anterior.

Iter.	V.Actual	dist[A]	dist[B]	dist[C]	dist[D]	dist[E]	dist[F]
Inicio	—	0	∞	∞	∞	∞	∞
1	**A**	0	**4**	**2**	∞	∞	∞
2	**C**	0	**3**	2	∞	**12**	∞
3	**B**	0	3	2	**8**	12	∞
4	**D**	0	3	2	8	**11**	**10**
5	**F**	0	3	2	8	11	10
6	**E**	0	3	2	8	11	10

Paso 4: Evolución del Arreglo de Predecesores pred[]

Iter.	V.Actual	pred[A]	pred[B]	pred[C]	pred[D]	pred[E]	pred[F]
Inicio	—	null	null	null	null	null	null
1	**A**	null	**A**	**A**	null	null	null
2	**C**	null	**C**	A	null	**C**	null
3	**B**	null	C	A	**B**	C	null
4	**D**	null	C	A	B	**D**	**D**
5	**F**	null	C	A	B	D	D
6	**E**	null	C	A	B	D	D

Paso 5: Reconstrucción del Camino Más Corto

Usando el arreglo pred[], reconstruimos el camino óptimo desde F hasta A y luego lo invertimos:

Paso	Vértice actual	pred[v]	Camino parcial
1	F	pred[F] = D	F
2	D	pred[D] = B	D <- F
3	B	pred[B] = C	B <- D <- F

Paso	Vértice actual	pred[v]	Camino parcial
4	C	pred[C] = A	C <- B <- D <- F
5	A	pred[A] = null (origen)	A <- C <- B <- D <- F
Inverso	**—**	**Camino final:**	**A -> C -> B -> D -> F**

Resultado Final

Camino mas corto de A a F:

A -> C -> B -> D -> F

Distancia total = 2 + 1 + 5 + 2 = 10

A->C: 2, C->B: 1, B->D: 5, D->F: 2 | costo total = 10

Paso 6: Análisis Detallado de Cada Iteración

Iteracion 1 - Vertice seleccionado: A (dist = 0)

Se selecciona A (dist = 0, es el origen). Se exploran los vecinos de A:

```
Vecino B: dist[A] + w(A,B) = 0 + 4 = 4 < INF  -> dist[B] = 4,  pred[B] = A
(relajacion exitosa)
Vecino C: dist[A] + w(A,C) = 0 + 2 = 2 < INF  -> dist[C] = 2,  pred[C] = A
(relajacion exitosa)
A queda marcado como visitado.
```

Iteracion 2 - Vertice seleccionado: C (dist = 2)

Se selecciona C (dist = 2, menor no visitado). Se exploran los vecinos de C:

```
Vecino A: dist[C] + w(C,A) = 2 + 2 = 4 >= dist[A] = 0  -> No se actualiza (A
ya visitado)
Vecino B: dist[C] + w(C,B) = 2 + 1 = 3 < dist[B] = 4  -> dist[B] = 3,  pred[B]
= C  (mejora)
Vecino E: dist[C] + w(C,E) = 2 + 10 = 12 < INF  -> dist[E] = 12,  pred[E] = C
(relajacion exitosa)
C queda marcado como visitado.
```

Iteracion 3 - Vertice seleccionado: B (dist = 3)

Se selecciona B (dist = 3, menor no visitado). Se exploran los vecinos de B:

```
Vecino A: dist[B] + w(B,A) = 3 + 4 = 7 >= dist[A] = 0  -> No se actualiza
Vecino C: dist[B] + w(B,C) = 3 + 1 = 4 >= dist[C] = 2  -> No se actualiza
Vecino D: dist[B] + w(B,D) = 3 + 5 = 8 < INF  -> dist[D] = 8,  pred[D] = B
(relajacion exitosa)
B queda marcado como visitado.
```

Iteracion 4 - Vertice seleccionado: D (dist = 8)

Se selecciona D (dist = 8, menor no visitado). Se exploran los vecinos de D:

```
Vecino B: dist[D] + w(D,B) = 8 + 5 = 13 >= dist[B] = 3  -> No se actualiza
Vecino E: dist[D] + w(D,E) = 8 + 3 = 11 < dist[E] = 12  -> dist[E] = 11,
pred[E] = D  (mejora)
```

```
Vecino F: dist[D] + w(D,F) = 8 + 2 = 10 < INF  -> dist[F] = 10,  pred[F] = D
(relajacion exitosa)
D queda marcado como visitado.
```

Iteracion 5 - Vertice seleccionado: F (dist = 10)

Se selecciona F (dist = 10, menor no visitado). Se exploran los vecinos de F:

```
Vecino D: dist[F] + w(F,D) = 10 + 2 = 12 >= dist[D] = 8  -> No se actualiza
Vecino E: dist[F] + w(F,E) = 10 + 1 = 11 = dist[E] = 11  -> Empate, no se
actualiza
F queda marcado como visitado. [DESTINO ALCANZADO] Distancia minima
confirmada: 10
```

Iteracion 6 - Vertice seleccionado: E (dist = 11)

Se selecciona E (dist = 11, ultimo no visitado). Los vecinos ya fueron visitados o no mejoran:

```
Vecino C: dist[E] + w(E,C) = 11 + 10 = 21 >= dist[C] = 2  -> No se actualiza
Vecino D: dist[E] + w(E,D) = 11 + 3 = 14 >= dist[D] = 8   -> No se actualiza
Vecino F: dist[E] + w(E,F) = 11 + 1 = 12 >= dist[F] = 10  -> No se actualiza
E queda marcado como visitado. Algoritmo terminado: todos los vertices
visitados.
```

Pseudocódigo

El siguiente pseudocódigo implementa el algoritmo de Dijkstra con las convenciones estándar para estructuras de control, arreglos y subprocesos.

CaminoMasCorto.pseint | Periodo 202660

```
// ================================================================
// ALGORITMO: Dijkstra - Camino Mas Corto
// Materia: Estructura de Datos | Periodo: 202660
// ================================================================

Algoritmo CaminoMasCorto
    Definir INF, N Como Entero
    INF <- 999999
    N   <- 6   // Vertices: A=0, B=1, C=2, D=3, E=4, F=5
    Definir dist, pred Como Entero
    Dimension dist[N]
    Dimension pred[N]
    Definir visitado Como Logico
    Dimension visitado[N]
    Definir mat Como Entero
    Dimension mat[N][N]

    // Inicializar matriz de adyacencia con INF
    Para i <- 0 Hasta N-1 Hacer
        Para j <- 0 Hasta N-1 Hacer
            mat[i][j] <- INF
        FinPara
    FinPara

    // Cargar aristas (A=0,B=1,C=2,D=3,E=4,F=5)
    mat[0][1] <- 4 ; mat[1][0] <- 4   // A-B: 4
    mat[0][2] <- 2 ; mat[2][0] <- 2   // A-C: 2
    mat[1][2] <- 1 ; mat[2][1] <- 1   // B-C: 1
    mat[1][3] <- 5 ; mat[3][1] <- 5   // B-D: 5
```

```
    mat[2][4] <- 10; mat[4][2] <- 10  // C-E: 10
    mat[3][4] <- 3 ; mat[4][3] <- 3   // D-E: 3
    mat[3][5] <- 2 ; mat[5][3] <- 2   // D-F: 2
    mat[4][5] <- 1 ; mat[5][4] <- 1   // E-F: 1

    Dijkstra(mat, dist, visitado, pred, 0, N, INF)
    MostrarCamino(pred, 5, N)
FinAlgoritmo

// ------------------------------------------------------------------
SubProceso Dijkstra(mat, dist, visitado, pred, origen, N, INF)
    Definir i, j, u, minDist Como Entero
    Para i <- 0 Hasta N-1 Hacer
        dist[i]     <- INF
        visitado[i] <- Falso
        pred[i]     <- -1
    FinPara
    dist[origen] <- 0

    Para i <- 0 Hasta N-1 Hacer
        // Seleccionar vertice u con menor distancia no visitado
        u <- -1 ; minDist <- INF
        Para j <- 0 Hasta N-1 Hacer
            Si (visitado[j] = Falso) Y (dist[j] < minDist) Entonces
                minDist <- dist[j]
                u <- j
            FinSi
        FinPara
        Si u = -1 Entonces FinPara FinSi
        visitado[u] <- Verdadero
        Escribir "Visitando: ", u, " dist=", dist[u]

        // Relajacion de aristas
        Para j <- 0 Hasta N-1 Hacer
            Si (visitado[j]=Falso) Y (mat[u][j] < INF) Entonces
                Si dist[u] + mat[u][j] < dist[j] Entonces
                    dist[j] <- dist[u] + mat[u][j]
                    pred[j] <- u
                FinSi
            FinSi
        FinPara
    FinPara
FinSubProceso

// ------------------------------------------------------------------
SubProceso MostrarCamino(pred, destino, N)
    Definir camino Como Entero
    Dimension camino[N]
    Definir k, actual Como Entero
    k <- 0 ; actual <- destino
    Mientras actual <> -1 Hacer
        camino[k] <- actual
        k      <- k + 1
        actual <- pred[actual]
    FinMientras
    Escribir "Camino optimo:"
    Para i <- k-1 Hasta 0 Con Paso -1 Hacer
        Si i > 0 Entonces
            Escribir camino[i], " -> ";
        Sino
            Escribir camino[i]
        FinSi
    FinPara
FinSubProceso
```

Salida Esperada en Consola

```
=== Distancias minimas desde A ===
  A -> A = 0
  A -> B = 3
  A -> C = 2
  A -> D = 8
  A -> E = 11
  A -> F = 10

Camino optimo: A -> C -> B -> D -> F
Distancia total: 10
```